A Technical Introduction to Federated learning

Section 1

HyFDCA is a provably convergent primal-dual algorithm for hybrid FL in at least the following settings. Hybrid Federated Setting with Complete Client Participation Horizontal Federated Setting with Random Subsets of Available Clients The authors show HyFDCA enjoys a convergence rate of O(1⁄t) which matches the convergence rate of FedAvg (see below). Vertical Federated Setting with Incomplete Client Participation The authors show HyFDCA enjoys a convergence rate of O(log(t)⁄t) whereas FedBCD exhibits a slower O(1⁄sqrt(t)) convergence rate and requires full client participation. HyFDCA provides the privacy steps that ensure privacy of client data in the primal-dual setting. These principles apply to future efforts in developing primal-dual algorithms for FL. HyFDCA empirically outperforms HyFEM and FedAvg in loss function value and validation accuracy across a multitude of problem settings and datasets (see below for more details). The authors also introduce a hyperparameter selection framework for FL with competing metrics using ideas from multiobjective optimization. There is only one other algorithm that focuses on hybrid FL, HyFEM proposed by Zhang et al. (2020). This algorithm uses a feature matching formulation that balances clients building accurate local models and the server learning an accurate global model. This requires a matching regularizer constant that must be tuned based on user goals and results in disparate local and global models. Furthermore, the convergence results provided for HyFEM only prove convergence of the matching formulation not of the original global problem. This work is substantially different than HyFDCA's approach which uses data on local clients to build a global model that converges to the same solution as if the model was trained centrally. Furthermore, the local and global models are synchronized and do not require the adjustment of a matching parameter between local and global models. However, HyFEM is suitable for a vast array of architectures including deep learning architectures, whereas HyFDCA is designed for convex problems like logistic regression and support vector machines. HyFDCA is empirically benchmarked against the aforementioned HyFEM as well as the popular FedAvg in solving convex problem (specifically classification problems) for several popular datasets (MNIST, Covtype, and News20). The authors found HyFDCA converges to a lower loss value and higher validation accuracy in less overall time in 33 of 36 comparisons examined and 36 of 36 comparisons examined with respect to the number of outer iterations. Lastly, HyFDCA only requires tuning of one hyperparameter, the number of inner iterations, as opposed to FedAvg (which requires tuning three) or HyFEM (which requires tuning four). In addition to FedAvg and HyFEM being quite difficult to optimize hyperparameters in turn greatly affecting convergence, HyFDCA's single hyperparameter allows for simpler practical implementations and hyperparameter selection methodologies.

Section 2

where K {\displaystyle K} is the number of nodes, x i {\displaystyle \mathbf {x} _{i}} are the weights of model as viewed by node i {\displaystyle i} , and f i {\displaystyle f_{i}} is node i {\displaystyle i} 's local objective function, which describes how model weights x i {\displaystyle \mathbf {x} _{i}} conforms to node i {\displaystyle i} 's local dataset. The goal of federated learning is to train a common model on all of the nodes' local datasets, in other words:

Section 3

A further governance-related issue concerns the distribution of value among participants as multi-party federated learning systems scale. As more clients contribute to training, model accuracy may approach a performance threshold, after which additional contributions yield diminishing marginal utility. This dynamic can create tensions within collaborative arrangements, as early participants may perceive later contributions as offering reduced value while still benefiting from the shared model. Such asymmetries may affect incentives to join or remain in a consortium and complicate the design of fair participation, ownership, or reward structures. Establishing transparent rules for contribution assessment and benefit allocation is therefore an important organisational challenge and represents a direction for future research.

Section 4

f (x 1 , … , x K) = 1 K ∑ i = 1 K f i (x i) {\displaystyle f(\mathbf {x} _{1},\dots ,\mathbf {x} _{K})={\dfrac {1}{K}}\sum _{i=1}^{K}f_{i}(\mathbf {x} _{i})}

Section 5

Hybrid federated dual coordinate ascent (HyFDCA) Very few methods for hybrid federated learning, where clients only hold subsets of both features and samples, exist. Yet, this scenario is very important in practical settings. Hybrid Federated Dual Coordinate Ascent (HyFDCA) is a novel algorithm proposed in 2024 that solves convex problems in the hybrid FL setting. This algorithm extends CoCoA, a primal-dual distributed optimization algorithm introduced by Jaggi et al. (2014) and Smith et al. (2017), to the case where both samples and features are partitioned across clients. HyFDCA claims several improvement over existing algorithms:

Section 6

Definition Federated learning aims at training a machine learning algorithm, for instance deep neural networks, on

multiple local datasets contained in local nodes without explicitly exchanging data samples. The general principle consists in training local models on local data samples and exchanging parameters (e.g. the weights and biases of a deep neural network) between these local nodes at some frequency to generate a global model shared by all nodes. The main difference between federated learning and distributed learning lies in the assumptions made on the properties of the local datasets, as distributed learning originally aims at parallelizing computing power where federated learning originally aims at training on heterogeneous datasets. While distributed learning also aims at training a single model on multiple servers, a common underlying assumption is that the local datasets are independent and identically distributed (i.i.d.) and roughly have the same size. None of these hypotheses are made for federated learning; instead, the datasets are typically heterogeneous and their sizes may span several orders of magnitude. Moreover, the clients involved in federated learning may be unreliable as they are subject to more failures or drop out since they commonly rely on less powerful communication media (i.e. Wi-Fi) and battery-powered systems (i.e. smartphones and IoT devices) compared to distributed learning where nodes are typically datacenters that have powerful computational capabilities and are connected to one another with fast networks.

Section 7

Initialization: according to the server inputs, a machine learning model (e.g., linear regression, neural network, boosting) is chosen to be trained on local nodes and initialized. Then, nodes are activated and wait for the central server to give the calculation tasks. Client selection: a fraction of local nodes are selected to start training on local data. The selected nodes acquire the current statistical model while the others wait for the next federated round. Configuration: the central server orders selected nodes to undergo training of the model on their local data in a pre-specified fashion (e.g., for some mini-batch updates of gradient descent). Reporting: each selected node sends its local model to the server for aggregation. The central server aggregates the received models and sends back the model updates to the nodes. It also handles failures for disconnected nodes or lost model updates. The next federated round is started returning to the client selection phase. Termination: once a pre-defined termination criterion is met (e.g., a maximum number of iterations is reached or the model accuracy is greater than a threshold) the central server aggregates the updates and finalizes the global model. The procedure considered before assumes synchronized model updates. Recent federated learning developments introduced novel techniques to tackle asynchronicity during the training process, or training with dynamically varying models. Compared to synchronous approaches where local models are exchanged once the computations have been performed for all layers of the neural network, asynchronous ones leverage the properties of neural networks to exchange model updates as soon as the computations of a certain layer are available. These techniques are also commonly referred to as split learning and they can be applied both at training and inference time regardless of centralized or decentralized federated learning settings.